

AI Customer Support Chatbot Builder

Documentation Suite — PRD · SRS · System Design · Database Design

A SaaS platform where any business uploads its FAQ and gets an AI-powered chat widget — built on a fully free tech stack (HuggingFace, Groq, Supabase) — that it can embed on its website within minutes.

Prepared for development handoff

Version 1.0 · June 2026 · Status: Draft for Build

Contains: Product Requirements · Software Requirements · System Design · Database Design

Table of Contents

Part 1: Product Requirements Document (PRD)

4

1.1 Executive Summary

4

1.2 Problem Statement

4

1.3 Goals and Success Metrics

4

1.4 Target Users and Personas

5

1.5 Core Feature List

5

1.6 User Flow — Business Owner

6

1.7 User Flow — End Website Visitor

6

1.8 MVP Scope and Roadmap

6

1.9 Monetization and Pricing Tiers

7

1.10 Competitive Landscape

7

1.11 Assumptions and Constraints

7

Part 2: Software Requirements Specification (SRS)

8

2.1 Purpose and Scope

8

2.2 Functional Requirements

8

2.3 Non-Functional Requirements

8

2.4 Security Requirements

9

2.5 External Interface Requirements

9

2.6 API Specification

10

2.7 Constraints (Free-Tier Limits)

10

2.8 Acceptance Criteria (Samples)

11

Part 3: System Design Document

12

3.1 Architecture Overview

12

3.2 Component Responsibilities

12

3.3 Ingestion Pipeline (Detailed)

13

3.4 Query Pipeline (Detailed)

13

3.5 Deployment Architecture

13

3.6 Scalability and Free-Tier Limitations

14

3.7 Reliability and Fallback Handling

14

3.8 Recommended Build Order

14

Part 4: Database Design Document

15

4.1 Entity Relationship Overview

15

4.2 Table Schemas

15

4.3 Indexes and Performance

16

4.4 Data Retention Considerations

17

4.5 Sizing Estimate on the Free Tier

17

PART 1

Product Requirements Document (PRD)

1.1 Executive Summary

The AI Customer Support Chatbot Builder is a self-serve SaaS product that lets any small business turn its existing FAQ content into a working, embeddable AI chat widget in minutes, with no coding and no developer involvement. A business owner pastes their FAQ text, the system automatically chunks and embeds that content, and visitors to the business's website can immediately start asking questions that are answered using only that business's own content — never invented information.

The defining constraint of this build is cost: the entire MVP runs on free-tier infrastructure. HuggingFace sentence-transformers provide embeddings at zero cost, Groq supplies fast, free Llama 3 inference, and Supabase provides both the relational database and the vector store (via pgvector) in a single free Postgres instance. This means the product can be built, launched, and operated for real businesses before a single rupee of infrastructure spend is required — payment processing via Razorpay is only introduced once the product needs to convert free users into paying ones.

1.2 Problem Statement

Small businesses cannot realistically staff round-the-clock live chat support. Customers expect instant answers to repetitive questions (return policy, pricing, hours, shipping, how something works) at any hour, but hiring even a part-time support agent is not viable for most small operations. Existing chatbot platforms solve this in theory, but in practice they are priced for mid-size and enterprise teams (\$20–\$100+/month) and frequently require technical setup (API keys, webhook configuration, custom integrations) that a non-technical small business owner cannot complete alone.

This product closes that gap with a self-serve flow built entirely on free AI infrastructure: paste your FAQ in, copy a script tag out, and the website has a 24/7 support assistant that only ever answers from the business's own content.

1.3 Goals and Success Metrics

Goal	Target Metric (MVP)
Fast onboarding	Time from signup to a live, working widget under 5 minutes
Answer accuracy	Bot honestly declines ("I don't know") rather than hallucinating when no relevant FAQ chunk is found, on 100% of out-of-scope questions tested
Low operating cost	\$0 infrastructure cost per business until paid-tier limits are introduced
Response speed	Visitor receives a generated answer in under 3 seconds end-to-end (helped by Groq's inference speed)
Adoption	Number of bots created, widgets actually embedded live on a business website, and total visitor queries answered per week

1.4 Target Users and Personas

Persona	Profile	Primary Need
Solo ecommerce owner	Runs a small online store, handles support personally between orders	Stop answering the same shipping/returns questions over and over, especially outside business hours
Local service business	Clinic, salon, repair shop, agency — mostly offline business with a basic website	A simple way to answer “do you offer X / what are your hours / how much does Y cost” without a phone line
Small SaaS support lead	Manages support for an early-stage software product, no budget for Intercom/Zendesk-tier tools	Deflect repetitive setup/billing questions from a small support inbox

1.5 Core Feature List

The table below is the authoritative feature list for the product, with tier availability and MVP status. Features marked Post-MVP are explicitly out of scope for the first build.

Feature	Description	Tier	MVP?
Business signup / login	Email-based account creation and authentication via Supabase Auth	All tiers	Yes
Create bot	Each business can create one or more chatbots	Free: 1 bot · Paid: 3 or unlimited	Yes (1-bot limit only)
FAQ upload (paste text)	Paste raw FAQ text into a textarea	All tiers	Yes
FAQ upload (file)	Upload a .txt file (later: .pdf)	All tiers	Post-MVP
Auto-chunking	System automatically splits FAQ content into retrievable chunks	Backend only	Yes
Auto-embedding	Each chunk is converted into a numeric vector for semantic search	Backend only	Yes
Retrain / re-upload	Replace old FAQ content with new content	All tiers	Yes (delete+replace)
Get embed snippet	Copy-paste JavaScript code for the business's website	All tiers	Yes
Live chat widget	Floating chat bubble that appears on the business's website	All tiers	Yes
AI-powered answers	Widget answers visitor questions using only the uploaded FAQ content	All tiers	Yes
“I don't know” fallback	Bot admits when it lacks information instead of hallucinating	All tiers	Yes
Query logs	Business owner can see what visitors have asked	All tiers	Yes (basic list)
Usage limit enforcement	Blocks bot creation or usage once a plan's limit is reached	Free / Basic	Post-MVP
Plan upgrade	Razorpay-powered payment to move to a paid tier	—	Post-MVP
Multi-bot dashboard	List and manage all bots from a single screen	Paid tiers	Post-MVP

1.6 User Flow — Business Owner

1. Owner visits the platform and signs up using email, handled by Supabase Auth.
2. Lands on the dashboard; sees a prompt to “Create your first bot” if no bots exist yet.
3. Clicks “Create Bot” and gives it a name (for example, “Support Bot”).
4. Is taken to the bot’s detail page, where they paste FAQ text into a textarea (or, post-MVP, upload a .txt file).
5. Clicks “Train Bot” and sees a loading state while the backend chunks, embeds, and stores the content.
Edge case: if the pasted text is empty or below a minimum length, the UI blocks submission with an inline message.
6. Once complete, sees a success message along with the embed snippet (a script tag containing a unique widget key) and a “Copy” button.
7. Pastes that snippet into their own website’s HTML, typically just before the closing `</body>` tag.
8. Can return at any time to view query logs, re-upload or retrain the FAQ (with a confirmation step, since retraining deletes and replaces all existing chunks for that bot), create additional bots if their plan allows, or upgrade their plan when they hit a limit.

1.7 User Flow — End Website Visitor

1. Visitor lands on the business’s website (for example, an online store).
2. Sees a floating chat bubble in the corner of the screen — the widget.
3. Clicks it, and a small chat panel opens.
4. Types a question (for example, “What’s your return policy?”).
5. The question is sent to the backend behind the scenes, identified only by the bot’s public widget key.
6. The backend finds the most relevant FAQ chunks and sends them, along with the question, to Groq.
7. Groq’s generated answer is returned and displayed in the chat panel within a couple of seconds.
8. The visitor can continue asking follow-up questions in the same session.
9. If the bot has no relevant information for a question, it says so honestly rather than inventing an answer.
Edge case: if the backend or an upstream AI service is temporarily unavailable, the widget shows a friendly “try again in a moment” message instead of failing silently.

1.8 MVP Scope and Roadmap

MVP Must-Haves

- Signup and login
- One bot per business
- Paste-text FAQ upload only (no file upload yet)
- Chunk, embed, and store pipeline
- Working chat widget with retrieval and Groq-generated answers
- Basic query log view

Explicitly Deferred to Post-MVP

- File upload (.txt / .pdf parsing)

- Multi-bot support
- Plan limit enforcement
- Razorpay integration
- Any analytics beyond a raw log list
- Retrain or versioning of FAQ content beyond simple delete-and-replace

1.9 Monetization and Pricing Tiers

Suggested starting tiers below; exact numbers should be validated against real Groq/Supabase free-tier limits before public launch (see Section 2.7, Constraints).

Tier	Price	Bots	Monthly Queries	Notes
Free	\$0	1	Capped (e.g. 200–500)	MVP default tier; limited by free Groq/Supabase quotas
Basic	Paid (Razorpay)	3	Higher cap	Introduced post-MVP once usage limit enforcement ships
Pro	Paid (Razorpay)	Unlimited	Highest cap / priority	Targeted at agencies managing bots for multiple clients

1.10 Competitive Landscape

Established players such as Intercom Fin, Tidio, Chatbase, and Crisp offer comparable FAQ-bot functionality but are priced for funded teams, typically starting in the \$20–\$100+/month range with usage-based add-ons. This product’s differentiation is structural, not feature-based: by building the entire MVP on free-tier AI and database infrastructure, it can offer a genuinely free starting tier to small businesses that the competition cannot match without subsidizing losses.

1.11 Assumptions and Constraints

- Groq and HuggingFace free-tier rate limits may require a paid upgrade path once usage scales beyond a few thousand queries per day.
- MVP supports text-pasted FAQ content only; file parsing is post-MVP.
- MVP is English-language only; multilingual support is a future feature.
- Each business is assumed to have basic ability to paste a script tag into their website HTML, or use a page builder that supports custom scripts.

PART 2

Software Requirements Specification (SRS)

2.1 Purpose and Scope

This section translates the product requirements in Part 1 into specific, testable functional and non-functional requirements, security requirements, and the REST API contract the frontend, backend, and widget will be built against. It is written so that a developer (human or AI) can implement against it without needing to re-derive behavior from the PRD narrative.

2.2 Functional Requirements

ID	Requirement	Priority
FR-1	System shall allow a business to sign up and log in using email-based authentication.	MVP
FR-2	System shall allow an authenticated business to create exactly one bot on the Free tier.	MVP
FR-3	System shall accept pasted FAQ text via a textarea and store it for processing.	MVP
FR-4	System shall split submitted FAQ text into retrievable chunks (by Q&A pair or paragraph block).	MVP
FR-5	System shall generate a numeric embedding vector for every chunk using a HuggingFace sentence-transformer model.	MVP
FR-6	System shall persist each chunk, its embedding vector, and its owning bot ID in the database.	MVP
FR-7	System shall generate a unique, non-guessable public widget key for each bot upon creation.	MVP
FR-8	System shall provide a copyable JavaScript embed snippet containing the bot's widget key.	MVP
FR-9	System shall accept a visitor question via the widget, embed it, retrieve the closest matching chunks, and generate an answer using Groq.	MVP
FR-10	System shall respond with an explicit "I don't know" style answer when no chunk meets the similarity threshold, rather than allowing the LLM to invent an answer.	MVP
FR-11	System shall log every visitor question and the answer given, associated with the owning bot.	MVP
FR-12	System shall allow the business owner to view a list of logged queries for their bot.	MVP
FR-13	System shall allow the business owner to retrain a bot, which deletes all existing chunks for that bot and replaces them with newly submitted content.	MVP
FR-14	System shall support file upload (.txt, later .pdf) as an alternative to pasting text.	Post-MVP
FR-15	System shall allow a business to create more than one bot, gated by plan tier.	Post-MVP
FR-16	System shall enforce bot-count and query-count limits per plan tier and block actions that exceed them.	Post-MVP
FR-17	System shall process plan upgrade payments via Razorpay and update the business's plan tier on success.	Post-MVP
FR-18	System shall provide a multi-bot dashboard listing all bots owned by a business.	Post-MVP

2.3 Non-Functional Requirements

Performance

- End-to-end visitor query response time (question sent → answer displayed) should stay under 3 seconds under normal load, relying on Groq's low-latency inference.
- Embedding generation for a typical FAQ upload (under ~5,000 words) should complete within a few seconds during training.

Scalability

- The FastAPI backend should be stateless so additional instances can be added behind the hosting provider's load balancing without code changes.
- Vector search via pgvector should remain performant as chunk volume grows by using an appropriate index (see Part 4, Section 4.3).

Availability

- The product should clearly communicate degraded states (e.g. "temporarily unavailable") rather than fail silently, especially given free-tier hosting cold-start behavior (see Part 3, Section 3.6).

Usability

- A non-technical business owner should be able to go from signup to a working embeddable widget without reading documentation, in under 5 minutes.

Maintainability

- Chunking, embedding, retrieval, and answer-generation logic should be implemented as separable functions within FastAPI so any one AI provider (HuggingFace, Groq) can be swapped later without rewriting the pipeline structure.

2.4 Security Requirements

- Authentication shall use Supabase Auth with JWT-based session tokens; the dashboard and management API routes require a valid JWT.
- Each bot's public widget key shall be a long, random, non-guessable token used only to scope a visitor's chat session to that bot — it shall never grant access to dashboard data, logs, or other bots.
- The public chat endpoint shall be rate-limited per widget key and per IP address to prevent abuse and to protect free-tier Groq/HuggingFace quotas.
- All visitor input shall be treated as untrusted: sanitized before storage and before being placed into the LLM prompt, to reduce prompt-injection risk that could otherwise be used to make the bot reveal other bots' content or ignore the "only answer from FAQ content" instruction.
- Data shall be encrypted in transit via HTTPS/TLS on all endpoints, and at rest via Supabase's default Postgres encryption.
- Provider API keys (HuggingFace, Groq, Razorpay) shall be stored only as backend environment variables and shall never be exposed to the dashboard frontend or the widget JavaScript.
- CORS shall be configured narrowly: dashboard/management API routes accept requests only from the product's own frontend origin, while the public widget chat route deliberately allows cross-origin requests, since it must be callable from any business's website domain.

2.5 External Interface Requirements

Service	Purpose	Auth Method
Supabase (Postgres + Auth + pgvector)	Relational data, authentication, and vector storage/search	Service-role API key (backend only)
HuggingFace sentence-transformers	Generate embeddings for FAQ chunks and visitor questions	Run locally in backend runtime, or HF Inference API key
Groq API (Llama 3)	Generate the final natural-language answer from retrieved context	Groq API key (backend only)
Razorpay	Process plan-upgrade subscription payments (post-MVP)	Razorpay key/secret (backend only)

2.6 API Specification

All routes are served by the FastAPI backend. Management routes require a valid Supabase JWT; the widget chat route is public but scoped by widget key and rate-limited.

Method & Endpoint	Auth	Purpose
POST /api/auth/signup	Public	Create a business account (email + password) via Supabase Auth
POST /api/auth/login	Public	Authenticate and receive a JWT session token
POST /api/bots	JWT	Create a new bot (name); enforces 1-bot Free-tier limit
GET /api/bots	JWT	List all bots owned by the authenticated business
POST /api/bots/{bot_id}/train	JWT	Submit pasted FAQ text; triggers chunk → embed → store pipeline
POST /api/bots/{bot_id}/retrain	JWT	Delete existing chunks and replace with newly submitted FAQ content
GET /api/bots/{bot_id}/embed-snippet	JWT	Return the JavaScript embed snippet containing the bot's widget key
GET /api/bots/{bot_id}/logs	JWT	Return the query log (question, answer, timestamp) for this bot
POST /api/widget/{widget_key}/chat	Public, rate-limited	Receive a visitor question, run the retrieval + Groq pipeline, return the answer
POST /api/billing/upgrade	JWT (Post-MVP)	Initiate a Razorpay payment to move the business to a paid plan

2.7 Constraints (Free-Tier Limits)

Because the MVP is built entirely on free infrastructure, the following provider limits directly constrain product behavior and should be re-verified against current provider documentation before launch, since free-tier terms change over time.

- Groq free tier: request-per-minute and token-per-minute caps apply to the Llama 3 answer-generation step; high traffic may require request queueing or a paid Groq tier.
- HuggingFace: if using the hosted Inference API rather than running sentence-transformers locally in the backend, free-tier rate limits apply; running the embedding model locally inside the FastAPI runtime avoids this

entirely.

- Supabase free tier: roughly 500MB of database storage and a connection-pool cap, which bounds total FAQ chunk volume across all businesses until upgrade.
- Render free web service tier: the backend instance may spin down after a period of inactivity, causing a cold-start delay on the next request (see Part 3, Section 3.6).

2.8 Acceptance Criteria (Samples)

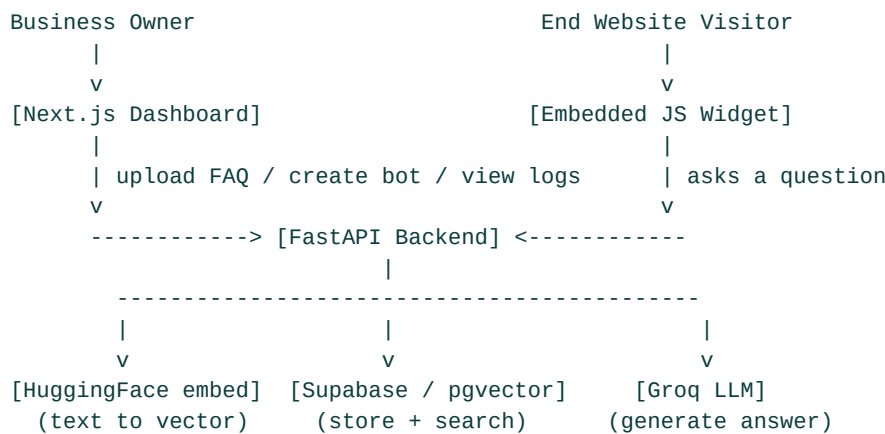
- **Given** a business has pasted valid FAQ text and clicked “Train Bot”, **when** training completes, **then** the dashboard displays a success state and a working embed snippet.
- **Given** a visitor asks a question covered by the FAQ, **when** the question is submitted, **then** the widget displays a relevant, FAQ-grounded answer within 3 seconds.
- **Given** a visitor asks a question with no relevant FAQ content, **when** the question is submitted, **then** the widget honestly states it does not have that information, with no invented answer.
- **Given** a business retrain a bot, **when** retraining completes, **then** all prior chunks for that bot are removed and only the newly submitted content is retrievable.

PART 3

System Design Document

3.1 Architecture Overview

Two distinct data flows run through the same FastAPI backend: the **ingestion flow**, which is owner-triggered and happens rarely (only on upload or retrain), and the **query flow**, which is visitor-triggered and happens constantly. Both pass through FastAPI, but the ingestion flow only touches HuggingFace and Supabase, while the query flow uses all three AI components together.



3.2 Component Responsibilities

Layer	Technology	Responsibility
Frontend framework	Next.js	Renders the dashboard and all business-facing pages: login, signup, bot management, embed snippet, logs
Backend API	FastAPI	Central brain: upload handling, chunking, embedding calls, vector retrieval, Groq calls, plan-limit checks
Database	Supabase (Postgres)	Stores businesses, bots, plan info, and query logs
Vector storage	pgvector (inside Supabase)	Stores embeddings and performs similarity search natively, removing the need for a separate vector DB
Embedding model	HuggingFace sentence-transformers (all-MiniLM-L6-v2)	Converts text to vectors, used in both ingestion and query pipelines
Answer-generation LLM	Groq API (Llama 3)	Converts retrieved context + question into a natural-language answer, used only in the query pipeline
Frontend hosting	Vercel	Free hosting tailored for Next.js
Backend hosting	Render	Free hosting for the FastAPI server and embedding model runtime
Payments	Razorpay	Collects subscription payments once a business upgrades (post-MVP only)
Widget delivery	Static JS file	A dependency-free script tag that lives entirely on the business's own website; only sends questions and displays answers

3.3 Ingestion Pipeline (Detailed)

Step	What Happens	Technical Detail
1	Owner submits FAQ text via the dashboard	Next.js POSTs to FastAPI /train endpoint
2	Backend receives the raw text	FastAPI request handler
3	Text is split into smaller chunks	By Q&A pair or paragraph block; target ~300–500 tokens per chunk with light overlap to preserve context across boundaries
4	Each chunk is converted into a numeric vector	HuggingFace all-MiniLM-L6-v2, 384-dimensional output
5	Each chunk, its vector, and its bot ID are saved together	Inserted into the documents table (Supabase, pgvector column)
6	A success response is sent back to the dashboard	FastAPI → Next.js. includes confirmation and chunk count

This pipeline only runs when a business uploads new content or retrains a bot — it never runs on a per-visitor-message basis.

3.4 Query Pipeline (Detailed)

Step	What Happens	Technical Detail
1	Visitor types a question into the widget	Widget JS
2	Widget sends the widget key and question to the backend	POST /api/widget/{widget_key}/chat
3	Backend looks up which bot this widget key belongs to	FastAPI → Supabase lookup on bots table
4	The question is converted into a numeric vector	HuggingFace all-MiniLM-L6-v2
5	That vector is compared against stored chunks for this bot	pgvector cosine similarity search, top-k (e.g. k=3–5), scoped to bot_id
6	Top matching chunks and the question are packaged into a prompt	FastAPI prompt template; if the best match is below a similarity threshold, skip straight to the fallback answer
7	The prompt is sent to Groq's Llama 3 model	Groq API call, instructed to answer only from supplied context
8	Groq returns a natural-language answer	Groq API → FastAPI
9	The exchange is logged and the answer is sent back	FastAPI → Supabase (queries_log insert), FastAPI → Widget JS
10	The widget displays the answer in the chat panel	Widget JS

3.5 Deployment Architecture

- Next.js dashboard deploys to Vercel's free tier, connected to the project's GitHub repository for automatic deploys on push.
- FastAPI backend deploys to Render's free web service tier, also connected to GitHub for automatic deploys.
- Supabase project hosts Postgres (with the pgvector extension enabled) and Supabase Auth; both are managed services requiring no separate hosting setup.
- All provider secrets (Groq key, HuggingFace key if used, Supabase service key, Razorpay keys) are stored as environment variables on Render, never committed to source control.

- The widget JS file is served as a static asset (from the backend or a CDN) and is the only artifact that physically lives on each business's own website.

3.6 Scalability and Free-Tier Limitations

Constraint	Free-Tier Behavior	Mitigation
Render cold start	Free web service instances spin down after inactivity; next request may take 30–50 seconds	Show a friendly loading state in the widget; consider a scheduled keep-alive ping or upgrade to a paid instance as usage grows
Groq rate limits	Free tier caps requests/tokens per minute	Add request queueing or backoff in FastAPI; monitor usage and upgrade before limits are regularly hit
Supabase storage cap	~500MB free Postgres storage bounds total chunk volume	Monitor table size; archive or upgrade before hitting the cap
HuggingFace Inference API limits	Rate-limited if using the hosted API	Run sentence-transformers locally inside the FastAPI runtime instead of calling a hosted API, avoiding this limit entirely

3.7 Reliability and Fallback Handling

- If the Groq API call fails or times out, the backend returns a graceful “temporarily unavailable, please try again” response rather than an error the widget cannot handle.
- If vector search returns no chunk above the similarity threshold, the backend short-circuits to the standard “I don’t know” response without calling Groq at all, saving a paid/rate-limited call.
- All backend exceptions are logged server-side; the widget only ever sees a small, well-defined set of response states (answer, fallback, temporary error).

3.8 Recommended Build Order

1. Prove the core AI loop works standalone — chunk, embed, store, retrieve, and get a Groq answer — before building any user interface.
2. Wrap that working loop in FastAPI endpoints.
3. Build and test the widget JS against those endpoints using a blank test page.
4. Build the Next.js dashboard: authentication, FAQ upload, embed snippet display, and logs.
5. Add multi-bot support and plan limit enforcement.
6. Add Razorpay integration and deploy publicly.

This order front-loads the riskiest, least familiar part of the project — AI retrieval quality — so problems surface in week one rather than after the entire interface has already been built around it. Dashboards and authentication are mechanical, lower-risk work that can be done with confidence once the core loop is proven.

PART 4

Database Design Document

4.1 Entity Relationship Overview

All persistent data lives in a single Supabase Postgres instance, with the `pgvector` extension enabled for embedding storage and similarity search. The core relationships are:

- One **business** can own many **bots** (1—N).
- One **bot** can have many **documents** (FAQ chunks) (1—N).
- One **bot** can have many **queries_log** entries (1—N).
- Deleting a bot should cascade-delete its documents and query logs, since orphaned chunks/logs serve no purpose.

4.2 Table Schemas

businesses

One row per signed-up business.

Field	Type	Notes
id	uuid, primary key	Matches Supabase Auth user id
email	text, unique, not null	Managed by Supabase Auth
plan_type	text, default 'free'	free · basic · pro
created_at	timestamptz, default now()	Signup timestamp

```
CREATE TABLE businesses (  
  id          uuid PRIMARY KEY REFERENCES auth.users(id),  
  email       text UNIQUE NOT NULL,  
  plan_type   text NOT NULL DEFAULT 'free',  
  created_at  timestamptz NOT NULL DEFAULT now()  
);
```

bots

One row per chatbot a business creates.

Field	Type	Notes
id	uuid, primary key, default <code>gen_random_uuid()</code>	Internal bot id
business_id	uuid, foreign key → <code>businesses.id</code>	Owning business
name	text, not null	Display name, e.g. 'Support Bot'
widget_key	text, unique, not null	Long random public token used by the embed snippet
created_at	timestamptz, default now()	Creation timestamp

```
CREATE TABLE bots (  
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
```

```

        business_id    uuid NOT NULL REFERENCES businesses(id) ON DELETE CASCADE,
        name           text NOT NULL,
        widget_key      text UNIQUE NOT NULL,
        created_at      timestampz NOT NULL DEFAULT now()
    );

```

documents

One row per FAQ chunk, with its embedding vector.

Field	Type	Notes
id	uuid, primary key, default gen_random_uuid()	Chunk id
bot_id	uuid, foreign key → bots.id	Owning bot
chunk_text	text, not null	The raw FAQ chunk content
embedding	vector(384)	all-MiniLM-L6-v2 output dimension
created_at	timestampz, default now()	Ingestion timestamp

```
CREATE EXTENSION IF NOT EXISTS vector;
```

```

CREATE TABLE documents (
    id            uuid PRIMARY KEY DEFAULT gen_random_uuid(),
    bot_id        uuid NOT NULL REFERENCES bots(id) ON DELETE CASCADE,
    chunk_text    text NOT NULL,
    embedding      vector(384),
    created_at    timestampz NOT NULL DEFAULT now()
);

```

queries_log

One row per visitor question asked.

Field	Type	Notes
id	uuid, primary key, default gen_random_uuid()	Log entry id
bot_id	uuid, foreign key → bots.id	Owning bot
question	text, not null	Visitor's raw question
answer	text	Answer given (or fallback message)
created_at	timestampz, default now()	Query timestamp

```

CREATE TABLE queries_log (
    id            uuid PRIMARY KEY DEFAULT gen_random_uuid(),
    bot_id        uuid NOT NULL REFERENCES bots(id) ON DELETE CASCADE,
    question      text NOT NULL,
    answer        text,
    created_at    timestampz NOT NULL DEFAULT now()
);

```

4.3 Indexes and Performance

- A unique btree index on bots.widget_key supports fast lookups on every visitor chat request.

- A btree index on documents.bot_id and queries_log.bot_id speeds up the common “all rows for this bot” queries.
- An IVFFlat (or HNSW, if the Supabase pgvector version supports it) index on documents.embedding accelerates cosine-similarity search as chunk volume grows; for small per-bot chunk counts at MVP scale, a plain sequential scan within a bot_id filter is also acceptable.

```
CREATE INDEX idx_bots_widget_key ON bots (widget_key);
CREATE INDEX idx_documents_bot_id ON documents (bot_id);
CREATE INDEX idx_queries_log_bot_id ON queries_log (bot_id);

-- Approximate nearest-neighbor index for cosine similarity search
CREATE INDEX idx_documents_embedding
  ON documents USING ivfflat (embedding vector_cosine_ops)
  WITH (lists = 100);
```

4.4 Data Retention Considerations

- Retraining a bot deletes all of its existing documents rows before inserting the newly submitted chunks, per FR-13.
- Deleting a bot cascades to remove its documents and queries_log rows, preventing orphaned data.
- queries_log can grow indefinitely under normal use; a future archival or rolling-retention policy (e.g. keep the last 90 days in the hot table) should be considered once usage scales, to stay within the Supabase free-tier storage cap.

4.5 Sizing Estimate on the Free Tier

Each all-MiniLM-L6-v2 embedding is 384 float values (~1.5KB uncompressed). With Supabase's ~500MB free Postgres allowance shared across all relational data, a rough back-of-envelope budget of 300MB dedicated to embeddings allows on the order of 150,000–200,000 FAQ chunks across all businesses before storage becomes a concern — comfortably enough headroom for the MVP and early post-launch period, with the remaining allowance covering business, bot, and log rows.

End of Document